

Что такое Scan и Seek? (Простая аналогия) Представьте, что база данных — это огромная библиотека, а таблица — это толстая энциклопедия. Seek (Поиск) — это когда вы открываете алфавитный указатель в конце книги, находите слово «Слон» и видите: «страница 450». Вы сразу открываете страницу 450. Вы сделали целенаправленный прыжок. Scan (Сканирование) — это когда вы ищете в книге цитату, но не помните, где она. Вам приходится читать книгу с первой страницы до последней. Вы читаете всё подряд. В MSSQL Seek ищет данные по индексу, а Scan просто читает всё подряд (всю таблицу или весь индекс).

В базах данных (особенно в MSSQL) всё крутится вокруг B-дерева.

Из чего строится дерево? (Страницы по 8 КБ) В MSSQL данные не лежат на диске сплошным неразборчивым куском. Они разбиты на кирпичики. Этот кирпичик называется Страница (Page), и её размер всегда строго 8 КБ. Индексы и таблицы строятся именно из этих 8-килобайтных страниц, которые связываются друг с другом в структуру дерева.

Структура B-дерева (B-Tree) Представьте себе перевернутое дерево. Оно состоит из трех уровней: Корневой уровень (Root Node) — это вершина. Всего одна страница (8 КБ). Это "директор". Промежуточный уровень (Intermediate Nodes) — "менеджеры". Они направляют поиск дальше. Листовой уровень (Leaf Nodes) — "работяги". Это самый нижний уровень, где лежат сами данные (или ссылки на них).

Строение узлов: что физически лежит внутри страниц? Мы помним, что всё дерево строится из Страниц (Pages) по 8 КБ..

Уровень 1: Корневой узел (Root Node) Это всегда одна страница. Внутри неё лежит простой массив пар значений: [Ключ] + [Указатель на следующую страницу].

Представьте, что внутри страницы Root лежит такая табличка: | Ключ (ID) | Указатель (Page ID) | Пояснение для нас | | :--- | :--- | :--- | | NULL | Страница № 100 | Всё, что меньше 500, ищи на стр. 100 | | 500 | Страница № 101 | Всё, что от 500 до 999, ищи на стр. 101 | | 1000 | Страница № 102 | Всё, что от 1000 и больше, ищи на стр. 102 | Как видите, здесь нет ни имен пользователей, ни их телефонов. Только ID и адреса других страниц.

Уровень 2: Промежуточные узлы (Intermediate Nodes) Их может быть много уровней (если таблица гигантская). MSSQL спустился, например, на Страницу №101. Внутри неё лежит точно такая же табличка, но с более мелкой нарезкой: Ключ (ID) | Указатель (Page ID) | Пояснение для нас | 500 | Страница № 550 | Всё от 500 до 699 — на стр. 550 | 700 | Страница № 551 | Всё от 700 до 899 — на стр. 551 | 900 | Страница № 552 | Всё от 900 до 999 — на стр. 552

Уровень 3: Листовые узлы (Leaf Nodes) — Самое важное! Здесь всё зависит от типа индекса (Кластерный или Некластерный). Если это Кластерный индекс (Clustered Index): На листовом уровне находится сама таблица. MSSQL приходит на Страницу №551 и видит там готовые строки. Если это Некластерный индекс (Non-Clustered Index): Мы искали, например, по индексу Фамилия. MSSQL спускается на листовую страницу и находит там: [Ключ: Смирнов] + [Указатель на то, где физически лежит строка Смирнова в главной таблице]

Важный момент: страницы листового уровня связаны между собой в цепочку (двусвязный список)

Два главных вида деревьев (Кластерный и Некластерный индекс). Деревья бывают двух видов: А) Кластерный индекс (Clustered Index) Это когда таблица сама по себе является деревом. На листовом уровне (внизу) лежат целиком все строки вашей таблицы со всеми колонками. Таблица отсортирована по ключу (обычно это колонка ID). Б) Некластерный индекс (Non-Clustered Index) Это отдельная, маленькая копия части ваших данных. Например, вы сделали некластерный индекс по колонке Фамилия. На листовом уровне этого дерева лежит только Фамилия и указатель (Pointer) на то, где в главной таблице лежит вся остальная строка (возраст, телефон, адрес).

4. Как под капотом работает SEEK (Прыжок по дереву) Представим запрос: `SELECT * FROM Users WHERE ID = 750`. У нас есть кластерный индекс по колонке ID. Как MSSQL ищет данные? MSSQL загружает в память Корневую страницу (Root). Там написано: Если ID от 1 до 500 — иди на промежуточную страницу №10. Если ID от 501 до 1000 — иди на промежуточную страницу №11. Нам нужен 750. MSSQL загружает страницу №11 (Intermediate). Там написано: Если ID от 501 до 800 — иди на листовую страницу №45. MSSQL загружает страницу №45 (Leaf). Это листовой уровень! Там он находит строку с ID = 750 и отдает ее вам. Итог: MSSQL прочитал с диска всего 3 страницы (24 Килобайта). Ему не важно, в таблице тысяча строк или миллиард. Из-за структуры дерева он всегда найдет нужную строку за 3-5 шагов. Это магия логарифмической сложности $O(\log N)$ $O(\log N)$.

5. Как под капотом работает SCAN (Сканирование) Представим другой запрос: `SELECT * FROM Users`. Нам нужны все пользователи. Что делает MSSQL? Он игнорирует корень дерева. Он игнорирует промежуточные уровни. Он идет в самый низ (на листовой уровень) к первой странице слева. Читает её. А так как листовые страницы связаны друг с другом цепочкой, он просто идет по этой цепочке: страница 1 → → страница 2 → → страница 3... и так до конца таблицы. Он читает данные "по горизонтали" сплошным потоком.

Главный вопрос: Что быстрее?

Правильный ответ: Всё зависит от количества строк, которые мы ищем.

Когда Seek рвет Scan? Когда нам нужна одна строка или пара десятков. Например: Найти заказ №155. MSSQL быстро пробежит по дереву (Seek), достанет заказ и закончит работу. Сканировать тут весь миллион заказов — глупо. Когда Scan рвет Seek? Представьте запрос: Найти все заказы со статусом "Отменен". И таких заказов в базе — половина от всего миллиона (500 000 штук). Если MSSQL решит использовать Seek по индексу «Статус», произойдет следующее: Он найдет в индексе 500 000 записей со статусом "Отменен". Но в индексе есть только статус! А нам для отчета нужны все данные (`SELECT *`). Значит, SQL Server должен 500 000 раз сбегать из индекса в основную таблицу, чтобы достать остальные колонки. Эта операция в MSSQL называется Key Lookup. Эти 500 000 прыжков — это случайное чтение (Random I/O). Жесткий диск или SSD сойдет с ума от постоянных метаний туда-сюда. Запрос будет висеть вечность. Поэтому, в таких случаях Оптимизатор запросов MSSQL поступает умнее. Он говорит: «Прыгать 500 000 раз слишком дорого. Я лучше сделаю Table Scan — просто прочитаю всю таблицу подряд от начала до конца единым потоком, и по пути выброшу ненужное». Это называется Tipping Point (Точка

перелома). Обычно, если запросу нужно достать больше 1-5% данных таблицы, MSSQL отказывается от Seek и переходит на Scan.

Seek идеален для точечного поиска (высокая селективность). Но он страдает от случайного чтения диска (Random I/O), если нужно сделать много прыжков (Key Lookups) в таблицу за недостающими данными. Scan идеален для аналитики, подсчета сумм или когда нужно вытащить большой кусок таблицы. Он выигрывает за счет быстрого последовательного чтения с диска (Sequential I/O). Оптимизатор MSSQL сам решает, что выбрать, основываясь на математике: что в данный момент будет стоить дешевле для диска и процессора».

Deferred-lock OCC

Гибридный OCC с отложенной блокировкой (или Прикладной оптимизм).

Самое простое объяснение того, как работает этот метод, можно свести к фразе: «Мы не делаем никаких скрытых копий, мы просто вешаем на строку счётчик правок и проверяем его перед тем, как сохранить изменения».

Вот как это работает на простом примере:

1. Как было в пессимистичном режиме (Блокировка)

Представь, что Оператор открывает карточку товара, чтобы изменить цену.

Пока он думает и вводит цифры, база намертво блокирует эту строку.

Другой Оператор в это время даже не может открыть эту карточку — у него всё «висит».

Это ужасно, если Оператор ушел пить кофе, оставив карточку открытой. Вся работа фирмы встала.

Подготовка (Что меняется в таблице)

В отличие от RCSI, здесь SQL Server ничего не делает сам. Ты (программист) добавляешь в таблицу обычную колонку, например Version (целое число) или UpdatedAt (время).

Когда строка создается, ей ставится Version = 1.

Это поле открыто и прозрачно, оно не скрытое.

2. Процесс изменения (Прикладной уровень)

Допустим, два человека — Вася и Петя — одновременно открыли один и тот же товар с ID = 1, у которого Price = 100 и Version = 5.

Чтение данных: И Вася, и Петя просто прочитали строку. База ничего не блокирует.

Оба видят в своих программах: «Цена 100, Версия 5».

Вася сохраняет первым: Он меняет цену на 110. Его программа отправляет в базу такой запрос:

```
UPDATE Products SET Price = 110, Version = 6 WHERE ID = 1 AND Version = 5;
```

Результат: База находит строку, где ID=1 и версия всё еще 5. Она обновляет её. Теперь в базе версия 6.

Петя сохраняет вторым: Он меняет цену на 120. Его программа (которая всё еще помнит старую версию 5) отправляет запрос:

```
UPDATE Products SET Price = 120, Version = 6 WHERE ID = 1 AND Version = 5;
```

Результат: База ищет строку, где ID=1 и версия 5... и не находит её! (Потому что Вася уже сделал её шестой).

Итог: Обновлено 0 строк. Петя получает сообщение: «Данные устарели, обновите страницу».

1. Этап Чтения (SELECT)

Когда ты читаешь строку, движок СУБД делает стандартный Index Seek (поиск по индексу).

В памяти приложения: Твоя программа (C#, Java, Python) запоминает не только саму цену, но и значение колонки Version.

В базе данных: В этот момент не происходит ничего особенного. Никаких копий не создается, никакие 14 байт не дописываются. База просто отдала данные и забыла про тебя.

2. Этап Записи (Тот самый "Под капот")

Когда ты отправляешь запрос UPDATE ... WHERE ID = 1 AND Version = 5, внутри движка SQL Server происходит следующая цепочка действий:

А) Поиск цели (Index Seek)

Движок ищет строку в индексе по двум ключам сразу: ID и Version.

Если Вася уже обновил строку, то в индексе больше нет записи, где Version = 5. Есть запись, где Version = 6.

SQL Server посмотрит в индекс и скажет: «Строка с такими параметрами не найдена».

Б) Кратковременная блокировка (Latch)

Даже если это «оптимистичный» метод, в момент физической записи данных на диск/в память движок всегда ставит очень короткую блокировку (она называется Latch или «защелка»).

Она нужна буквально на микросекунды, чтобы два процессора не попытались одновременно изменить одни и те же биты в памяти.

Это гарантирует атомарность: проверка версии и сама запись происходят как один неразрывный шаг.

В) Сверка результата (Row Count)

Это самый важный момент «под капотом». Движок SQL Server всегда возвращает количество строк, которые он затронул (@@ROWCOUNT).

Если версия совпала

→

SQL Server меняет данные и возвращает 1.

Если версия изменилась

→

SQL Server ничего не находит и возвращает 0.

Где хранятся данные?

Все данные хранятся прямо в твоей таблице.

Старые версии просто стираются и заменяются новыми.

База данных не тратит ресурсы на хранение истории изменений. Вся история «живет» только в логике твоего запроса.

Почему это называется "Deferred-lock" (Отложенная блокировка)?

Слово Deferred (отложенный) означает, что мы «откладываем» решение о блокировке до самого последнего момента.

В пессимистичном режиме мы блокируем строку в начале (когда только открыли карточку товара).

В этом режиме мы вообще не блокируем строку, пока пользователь думает. Мы блокируем её только на миллисекунду в самом конце, когда нажимаем кнопку «Сохранить», чтобы проверить версию и записать данные.

Какой подвох?

За то, что мы не загружаем tempdb и не блокируем таблицу, мы платим тремя вещами:

Ручная работа: Тебе нужно самому не забывать добавлять `Version = Version + 1` и проверку `WHERE Version = ...` в каждый `UPDATE`. Если забудешь один раз — данные будут затерты (аномалия «Потерянное обновление»).

Перезапуск вручную: Если произошел конфликт (как у Пети), транзакция не просто «подождала и прошла». Петя реально потерял то, что вводил. Ему нужно нажать «Обновить», увидеть цену Васи и вводить всё заново.

Лишние данные: В каждой твоей таблице появляется «техническая» колонка, которая не нужна бизнесу, но нужна только для контроля версий.